



دانشکده مهندسی کامپیوتر
مبانی و کاربردهای هوش مصنوعی ترم بهار ۱۴۰۴

پروژه دوم
مهلت تحویل ۵ اردیبهشت ساعت ۲۳:۵۹

مقدمه

در این پروژه قصد داریم سودوکو حل کنیم. ولی نه با کاغذ و مداد! به جای آن، از الگوریتم‌هایی که در طول درس با آن‌ها آشنا شدیم کمک می‌گیریم تا سودوکوهای مختلف را برایمان حل کند. قبل از آن نگاهی دوباره به مسئله‌ی ارضای محدودیت (CSP) می‌اندازیم:

مسئله‌ی ارضای محدودیت

هر مسئله‌ی ارضای محدودیت از یک سه تایی $(\mathcal{X}, D, C)$ تشکیل می‌شود که شامل متغیرها $(\mathcal{X})$ ، دامنه‌ی هر متغیر (D) و شرط‌ها (C) می‌شود. هدف از حل یک مسئله‌ی CSP، اختصاص مقادیر مشخصی از دامنه‌ی متغیرها به آن‌ها است به طوری که تمام شرط‌ها برقرار شوند.

برای اینکه سودوکو را به صورت یک مسئله‌ی ارضای محدودیت مدل کنیم باید سه تایی $(\mathcal{X}, D, C)$ را برای آن تعریف کنیم. این سه تایی به صورت زیر تعریف می‌شوند:

- $\mathcal{X}$: مجموعه‌ی تمام خانه‌های سودوکو. هر خانه را با x_{ij} نمایش می‌دهیم که i نماد سطر و j نماد ستون آن است. همچنین $1 \leq i, j \leq 9$ است.

- D : مقادیر بین ۱ تا ۹

$$\mathcal{C} = \begin{cases} x_{ij} = v_{ij} & \text{iff } v_{ij} > 0 \\ x_{ij} \neq x_{ik} & \text{iff } j \neq k \\ x_{ij} \neq x_{kj} & \text{iff } i \neq k \\ x_{ij} \neq x_{kl} & \text{iff } (i \neq k \vee j \neq l) \wedge \lceil \frac{i}{3} \rceil = \lceil \frac{k}{3} \rceil \wedge \lceil \frac{j}{3} \rceil = \lceil \frac{l}{3} \rceil \end{cases} : \mathcal{C} \bullet$$

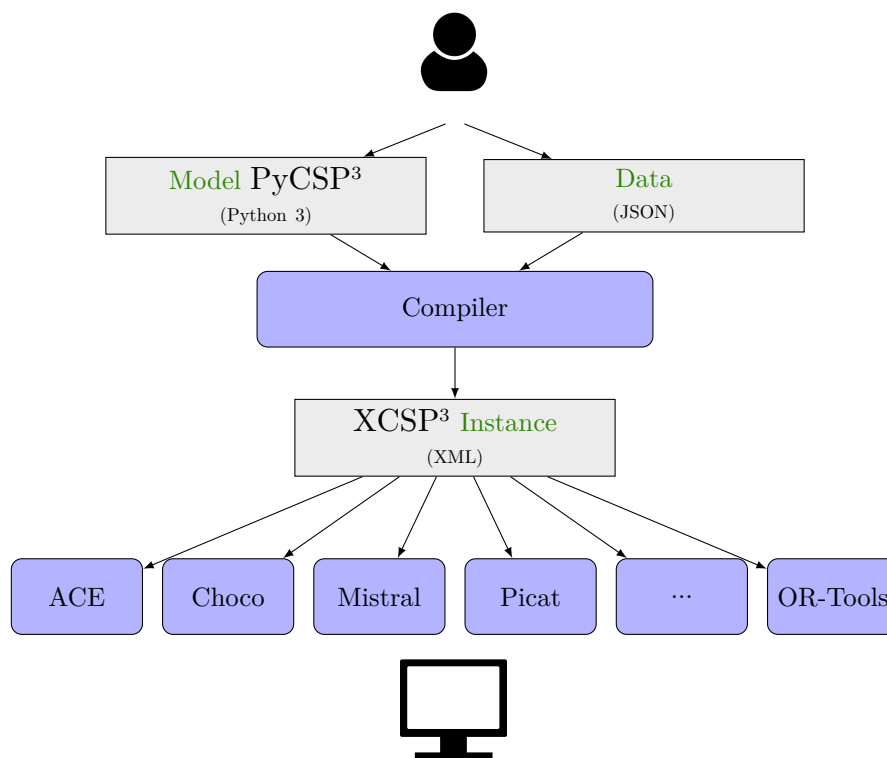
که $V = \{v_{ij}\}$ یک جدول ۹ در ۹ با مقادیر بین صفر تا ۹ است.

$V =$

		2						
8	9			4				
			5				1	
		5				7		
				8			3	
					9			
								8
					2		4	9
		6	7					

بخش‌های پروژه

این پروژه از دو بخش تشکیل شده است. در بخش اول با یکی از کتابخانه‌های پایتون آشنا می‌شویم که جهت حل این نوع مسائل نوشته شده است (با نام `pycsp`). و در بخش دوم سعی می‌کنیم خودمان شبه کتابخانه‌ای مانند `pycsp` بنویسیم (نام آن را `mycsp` می‌گذاریم). در ادامه با ساختار پروژه بیشتر آشنا می‌شویم و سودوکو را با این کتابخانه حل می‌کنیم.



solver هایی که استفاده می کنیم (ACE و Choco) با جاوا نوشته شده اند. لذا پیشنهاد می دهیم حتما **Java 11** یا بالاتر روی سیستم تان نصب باشد.

توجه: در آخرین نسخه های `pycsp3` ممکن است در هنگام استفاده از محیط های مجازی (مانند `venv` و `conda`) دچار مشکل شوید. در این صورت اگر ورژن پایتون سیستم شما پایین تر از `python3.10` است، ابتدا پایتون نسخه ی بالاتر (ترجیحا 3.11) را نصب کنید و سپس با دستور زیر با آن ورژن خاص کار کنید:

```
python3.11 -m ... // mac or linux
```

```
py -3.11 -m ... // windows
```

مثلا برای نصب کتابخانه یا اجرای برنامه دستورات زیر را اجرا کنید:

```
python3.11 -m pip install pycsp3 // mac or linux
```

```
py -3.11 -m main.py // windows
```

- `board.py`: کلاس `Board` در این فایل قرار دارد. هر صفحه ی سودوکو سه آرایه دارد که در این کلاس تعریف شده اند:

– `empty_board`: یک آرایه‌ی ۹ در ۹ که همه‌ی مقادیر آن صفر هستند. هنگامی که می‌خواهیم یکی از آرایه‌های `Board` را `reset` کنیم آن را برابر این مقدار قرار می‌دهیم.

– `layout_board`: مقادیر اولیه‌ی سودوکو در این آرایه `load` می‌شوند. این آرایه تا زمان حل سودوکو نباید تغییر کنند (این اعداد با رنگ سیاه نمایش داده خواهند شد).

– `guess_board`: در صورتی که الگوریتم شما بخواهد در لحظه مقادیر انتخاب شده‌اش برای متغیرها را روی صفحه نمایش دهد، باید آن اعداد را در این آرایه ذخیره کند. در هنگام `refresh` شدن گرافیک برنامه، مقادیر این آرایه خوانده می‌شوند (این اعداد با رنگ آبی نمایش داده خواهند شد).

– `answer_board`: زمانی که الگوریتم شما جواب نهایی را برای تمامی متغیرها پیدا کرد، باید آن مقادیر را در این آرایه بریزید (این اعداد با رنگ قرمز در هنگام حل سودوکو به طور کامل نمایش داده خواهند شد).

– `remaining_domains`: برعکس آرایه‌های قبلی که ۹ در ۹ بودند، این یک آرایه‌ی ۳ بعدی است که برای هر خانه‌ی سودوکو، لیستی از تمام مقادیر ممکن برای آن خانه را نگه می‌دارد (این اعداد به صورت اعداد کوچک و با رنگ خاکستری نمایش داده خواهند شد).

• `sudoku.py`: کلاس `Layout` در این فایل قرار دارد که مقادیر سودوکو را از داخل فایل خوانده و نگهداری می‌کند. با تابع `get_clues()` می‌توانید این اعداد را بخوانید. بعضی از توابع این فایل را باید پیاده سازی کنید.

• `refresher.py`: برای اینکه بتوانید در لحظه رفتار الگوریتم خودتان را مشاهده کنید، از کلاس `Refresher` که در این فایل تعریف شده است کمک بگیرید. هنگامی که می‌خواهید مقادیر تغییر یافته‌ی متغیرها و دامنه‌های آن‌ها را روی صفحه نمایش بدهید، بر روی شیء `refresher` که به عنوان ورودی الگوریتم می‌گیرید، تابع `refresh_screen()` را صدا بزنید تا مقادیر و دامنه‌های خانه‌های روی صفحه بروزرسانی شوند و روند اجرای الگوریتم خود را ببینید. با زدن دکمه‌ی `real-time` در رابط کاربری، می‌توانید این گزینه را خاموش یا روشن کنید.

• `exceptions.py`: استثناهای برنامه در این فایل نوشته شده‌اند.

- `example.py`: این فایل ربطی به فایل‌های دیگر پروژه ندارد و صرفاً مثالی از استفاده از کتابخانه‌ی `pycsp` در آن آورده شده است. می‌توانید تست‌های خود بر روی `pycsp` را در این فایل انجام دهید.

بخش اول: آشنایی با کتابخانه‌ی pycsp

`pycsp` یک کتابخانه‌ی متن‌باز^۱ پایتون است که برای حل مدل‌های^۲ گسسته‌ی ارضای محدودیت توسعه داده شده است. برای آشنایی بیشتر با این کتابخانه روی این [لینک](#) کلیک کنید.

برای استفاده از این کتابخانه ابتدا باید آن را نصب کنیم:

```
pip install pycsp3
```

توجه: برای استفاده از این کتابخانه به پایتون نسخه‌ی 3.10 یا بالاتر دارید. نحوه‌ی کار با این کتابخانه بسیار ساده است. یک نمونه از استفاده از آن در زیر آورده شده است:

```
1 from pycsp3 import *
2 x = VarArray(size=5, dom=range(5))
3 satisfy(
4     AllDifferent(x)
5 )
6 if solve() is SAT:
7     print(values(x))
```

در این قطعه کد، ابتدا متغیرها به صورت یک `VarArray` تعریف می‌شوند که متشکل از ۵ متغیر است و دامنه‌ی هر کدام از صفر تا چهار است. سپس شرط‌ها به تابع `satisfy()` داده می‌شود (مثلا در این مثال تنها شرط `AllDifferent` است؛ به این معنی که تمام متغیرهای آرایه‌ی `x` باید متفاوت باشند). و در آخر نیز تابع `solve()` صدا زده می‌شود و اگر خروجی `SAT` داد، به این معنی است که یک پاسخ برای این مسئله پیدا کرده است و با تابع `values()` می‌توان این مقادیر را خواند.

راهنمایی: برای اینکه با این کتابخانه بیشتر آشنا شوید، مسئله‌ی n وزیر به صورت یک مسئله‌ی CSP در فایل `example.py` نوشته شده است. آن را تحلیل کنید.

سوال ۱: توضیح دهید هرکدام از شرطهای n وزیر به چه صورت در این فایل نوشته شده‌اند.

پیاده سازی ۱: تابع `pycsp_solve()` داخل فایل `sudoku.py` را پیاده سازی کنید. در این تابع ابتدا باید مقادیر قبلی متغیرها و شرطها با دستور `clear()` پاک شوند (چون `pycsp` این مقادیر را به صورت `global` نگه می‌دارد و هر دفعه باید پاک شوند). سپس متغیرها، دامنه‌ی آنها و شرطهای آنها را تعریف کنید و در آخر با این کتابخانه مسئله‌ی سودوکویی که تعریف کرده‌اید را حل کنید. در آخر نیز اگر `pycsp` جوابی برای مسئله پیدا کرد، آرایه‌ی `board.answer_board` را برابر مقادیر بدست آمده قرار دهید. در صورت پیدا شدن جواب `True` و در غیر این صورت `False` برگردانید.

بعد از پیاده سازی، وقتی بر روی دکمه‌ی `solve` می‌زنید، بعد از چند لحظه باید جواب سودوکو به صورت زیر نمایش داده شود:

5	3	2	8	1	7	9	6	4
8	9	1	2	4	6	3	5	7
4	6	7	5	9	3	8	1	2
3	8	5	4	2	1	7	9	6
2	7	9	6	8	5	4	3	1
6	1	4	3	7	9	2	8	5
1	2	3	9	6	4	5	7	8
7	5	8	1	3	2	6	4	9
9	4	6	7	5	8	1	2	3

Solve

Load

pycsp ▾

Unary Checker

Arc Consistency

real-time

MRV

LCV

بخش دوم: پیاده‌سازی کتابخانه‌ی mycsp

در بخش قبل با کتابخانه‌ای آشنا شدیم که می‌تواند دامنه‌ی وسیعی از مسائل ارضای محدودیت را حل کند. در این بخش می‌خواهیم کتابخانه‌ای نسبتاً کوچک بنویسیم که بتواند خانواده‌ای از مسائل ارضای محدودیت را حل کند و سپس با آن سودوکو را حل کنیم. داخل پوشه‌ی `myCSP` چندین فایل وجود دارد که بخشی از این کتابخانه در آن‌ها پیاده‌سازی شده‌اند. وظیفه‌ی شما پیاده‌سازی توابع داخل فایل `mycsp.py` است. قبل از پیاده‌سازی آن‌ها، فایل‌های این پوشه را بررسی کنید (همه‌ی کدها کامنت کافی دارند).

پیاده‌سازی NODE CONSISTENCY

یک متغیر `node-consistent` است اگر و تنها اگر تمام مقادیر دامنه‌اش شرط‌های unary آن را برآورده کنند. در مسئله‌ی سودوکو، شرط‌های unary همان مقادیر اولیه‌ی خانه‌ها هستند که در ابتدا ذکر می‌شوند.

پیاده‌سازی ۲: تابع `node_consistency` در فایل `mycsp.py` را پیاده‌سازی کنید. هنگامی که دامنه‌ی خانه‌ای تغییر یافت، با شی `refresher` مقادیر صفحه را بروزرسانی کنید.

سوال ۲: چرا در تابع `solve()` ابتدا `node-consistency` انجام می‌شود؟ اگر این کار انجام نشود چه مشکلی به وجود می‌آید؟ بعد از اینکه بخش‌های بعدی را هم پیاده‌سازی کردید، سعی کنید سناریویی را به وجود بیاورید که وقتی یکی از سودوها را باز کردید، در حالی که دکمه‌ی `real-time` روشن است و بعضی از هیوریستیک‌ها را خاموش و بعضی را روشن نگه‌داشته‌اید، روشن یا خاموش کردن داکمه‌ی `unary checker` در زمان حل آن سودوکو تفاوت ایجاد کند. در پاسخ خود نام آن سودوکو، لیست دکمه‌های فعال و لیست دکمه‌های غیر فعال را ذکر کنید.

پیاده‌سازی BACKTRACK

ساده‌ترین روش حل یک مسئله‌ی CSP، استفاده از الگوریتم جستجوی اول عمق (DFS) است. اما الگوریتم BACKTRACKING-SEARCH با DFS تفاوت‌هایی دارد.

سوال ۳: یکی از تفاوت‌های DFS و BACKTRACKING-SEARCH را بیان کنید.

شبه‌کد BACKTRACKING-SEARCH در اسلایدهای درس موجود است. برای سادگی شما آن را آورده‌ایم.

Algorithm 1 Backtracking Search Algorithm

```
1: function BACKTRACKING-SEARCH(csp) returns a solution or failure
2:   return BACKTRACK(csp, {})

3: function BACKTRACK(csp, assignment) returns a solution or failure
4:   if assignment is complete then
5:     return assignment
6:   var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)                                ▷ #1
7:   for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do                ▷ #2
8:     if value is consistent with assignment then
9:       add {var = value} to assignment
10:    inferences ← INFERENCE(csp, var, assignment)
11:    if inferences ≠ failure then
12:      add inferences to csp
13:      result ← BACKTRACK(csp, assignment)
14:      if result ≠ failure then
15:        return result
16:      remove inferences from csp
17:      remove {var = value} from assignment
18:   return failure
```

پیاده‌سازی ۳: تابع `backtrack()` در فایل `mycsp.py` را پیاده‌سازی کنید. در پیاده‌سازی ممکن است نیاز به توابعی دیگر پیدا کنید، لذا توابع دیگر این فایل رو بررسی کنید و از آن‌ها استفاده کنید. توجه کنید که در زمان‌های تغییر مقادیر و دامنه‌های متغیرها، صفحه را `refresh()` کنید.

برای اینکه الگوریتم `backtrack()` خود را بتوانید مشاهده کنید، نیاز است بخش‌های #1 و #2 که در شبه‌کد بالا مشخص شده‌اند را پیاده‌سازی کرده باشید. ساده‌ترین هیوریستیک انتخاب متغیر، هیوریستیکی است که متغیرها را با ترتیب انتخاب می‌کند. همچنین ساده‌ترین هیوریستیک انتخاب مقادیر متغیرها، هیوریستیکی است که مقادیر را به ترتیب لیست می‌کنند و برمی‌گردانند.

پیاده‌سازی ۴: تابع `select_static_order_variable()` را در فایل `mycsp.py` پیاده‌سازی کنید. این تابع باید اولین متغیر که مقداری ندارد را برگرداند.

پیاده‌سازی ۵: تابع `static_order_domains()` را در فایل `mycsp.py` پیاده‌سازی کنید. این تابع باید لیست مقادیر باقی‌مانده از متغیر را به همان ترتیب که ذخیره شده است برگرداند.

استفاده از کتابخانه‌ی `mycsp`

اکنون که الگوریتم `backtrack()` شما دارای هیوریستیک‌های ابتدایی برای انتخاب متغیرها و دامنه‌ی آن‌ها برای مقدار دهی است، نیاز است که آن را تست کنیم. در بخش اول با استفاده از کتابخانه‌ی `pycsp` متغیرها، دامنه‌ها و شرط‌های سودوکو را تعریف کردیم و با آن سودوکو را حل کردیم. اکنون می‌خواهیم همینکار را با کتابخانه‌ی خودمان انجام دهیم.

پیاده‌سازی ۶: تابع `mycsp_solve()` داخل فایل `sudoku.py` را پیاده‌سازی کنید. برای تعریف متغیرها از `myVarArray` و برای تعریف شرط‌های بین خانه‌ها (شرط‌های binary) از `myAllDifferent` استفاده کنید. همچنین برای تعریف شرط‌های unary از `myUnaryConstraint` استفاده کنید. دستور `my_solve()` علاوه بر پارامترهای هیوریستیک‌ها، شیء `refresher` هم می‌گیرد که برای بروزرسانی صفحه استفاده می‌شود. لذا باید یک شیء `Refresher` ایجاد کنید و متغیرها (که یک آرایه‌ی دو بعدی است) و پارامترهای مورد نیازش را بدهید. در آخر هنگام صدا زدن `my_solve()` این شیء را تحویل دهید.

سوال ۴: شرط‌های موجود در سودوکو را به صورت `AllDifferent` تعریف کردیم که هر شرط برای ۹ متغیر تعریف می‌شود. این شرط‌ها چگونه به شرط‌های دوتایی (binary) تبدیل شده‌اند؟ آیا به طور کلی می‌توان هر شرط چند متغیره را به شرط‌های دو متغیره تبدیل کرد؟ اگر بله، چگونه؟

پیاده‌سازی ARC-CONSISTENCY

هنگامی که الگوریتم `backtrack()` را بدون هیوریستیک اجرا کردیم، دیدیم که حتی برای حل سودوکوهای آسان نیز زمان زیادی طول می‌کشد. اکنون می‌خواهیم با الگوریتم‌های forward checking با انتخاب مقدار یک متغیر، در صورتی که جوابی وجود نداشت زودتر باخبر شویم و `backtrack` کنیم. یکی از این روش‌ها استفاده از ARC-CONSISTENCY است که در درس خواندیم. این الگوریتم شرط‌های binary را بررسی می‌کند و مقادیری از متغیرها که در هیچ شرایطی آن شرط‌ها را برآورده نمی‌کنند را حذف می‌کند.

الگوریتم AC-3 یکی از پیاده‌سازی‌های ARC-CONSISTENCY است که در درس با آن آشنا شدیم و شبه‌کد آن را دیدیم که در زیر آورده شده است.

Algorithm 2 Arc-Consistency Algorithm

```
1: function AC-3(csp) returns false if an inconsistency is found and true otherwise
2:   queue  $\leftarrow$  a queue of arcs, initially all the arcs in csp
3:   while queue is not empty do
4:      $(X_i, X_j) \leftarrow \text{POP}(\text{queue})$ 
5:     if REVISE(csp,  $X_i$ ,  $X_j$ ) then
6:       if size of  $D_i = 0$  then
7:         return false
8:       for each  $X_k$  in  $X_i.\text{NEIGHBORS} - \{X_j\}$  do
9:         add  $(X_k, X_i)$  to queue
10:  return true

11: function REVISE(csp,  $X_i$ ,  $X_j$ ) returns true iff we revise the domain of  $X_i$ 
12:   revised  $\leftarrow$  false
13:   for each  $x$  in  $D_i$  do
14:     if no value  $y$  in  $D_j$  allows  $(x, y)$  to satisfy the constraint between  $X_i$  and  $X_j$  then
15:       delete  $x$  from  $D_i$ 
16:       revised  $\leftarrow$  true
17:  return revised
```

پیاده‌سازی ۷: توابع `arc_consistency()` و `revise` فایل `mycsp.py` را پیاده‌سازی کنید. هنگامی که دامنه‌ی یکی از متغیرها تغییر می‌کند، تابع `refresh_screen()` را صدا بزنید. برای گرفتن صف یال‌ها، از `g.get_arcs()` استفاده کنید. همچنین برای بررسی برقرار بودن یک شرط برای دو متغیر، از تابع

`g.is_arc_satisfied(v1, v2, x1, x2)`

استفاده کنید که در آن $v1$ و $v2$ متغیرها و $x1$ و $x2$ به ترتیب مقادیر آن‌ها هستند.

سوال ۵: سودوکوی `Without Solution.sudoku` را با `arc_consistency` حل

کنید. چرا بعد از امتحان کردن فقط چند مقدار، الگوریتم نتیجه گرفت که سودوکو جواب ندارد؟ اگر از arc-consistency استفاده نکنیم چقدر زمان می‌برد تا نتیجه بگیرد که جوابی وجود ندارد؟ این اختلاف زمان را توجیح کنید.

پیاده‌سازی MINIMUM-REMAINING-VALUES

تا الآن الگوریتم شما متغیرها را از اول انتخاب و مقدار دهی می‌کرد. در صورتی که ترتیب مقداردهی متغیرها را تغییر دهیم، می‌توانیم درخت جستجو را کوچک کنیم. انتخاب اول متغیرهایی که بیشتر از بقیه روی آن‌ها شرط اعمال شده است، باعث می‌شود ضریب انشعاب گره‌های بالایی درخت جستجو کاهش یابد و درخت کوچک شود. از آنجایی که تعداد شرط‌های تعریف شده برای هر متغیر در مسئله سودوکو ثابت است، متغیرهایی که کمترین تعداد دامنه را دارند گزینه‌ی مناسبی برای انتخاب‌های اول هستند.

سوال ۶: در گراف شرط‌های مسئله سودوکو، هر گره چند یال دارد؟

پیاده‌سازی ۷: تابع `minimum_remaining_values()` فایل `mycsp.py` را پیاده‌سازی کنید. این تابع باید متغیری که دامنه‌ی کوچک‌تری دارد را برگرداند. متغیر انتخاب شده نباید مقداردهی شده باشد (`v.value` باید `None` باشد).

پیاده‌سازی LEAST-CONSTRAINING-VALUE

وقتی که متغیری انتخاب می‌شود، الگوریتم باید ترتیب مقداردهی به آن را انتخاب کند. هیوریستیک LEAST-CONSTRAINING-VALUE برای این مقداردهی عملکرد خوبی دارد. این هیوریستیک مقداری را انتخاب می‌کند که در آینده شانس بیشتری برای جواب داشتن دارند. برای پیاده‌سازی آن، مقداری را ترجیح می‌دهیم که کمترین گزینه‌ها را برای متغیرهای همسایه در گراف محدودیت حذف کند.

پیاده‌سازی ۸: تابع `least_constraining_value()` فایل `mycsp.py` را پیاده‌سازی کنید. برای هر مقدار، ابتدا تعداد مقداری از همسایگان که حذف می‌شوند را بدست آورید. سپس آن‌ها را بر اساس تعداد مقادیر حذف شده مرتب کنید. مقداری که حذف شدگان‌شان بیشتر بودند باید آخر باشند. در آخر این لیست را برگردانید.

سوال ۷: سودوکوی `Medium.sudoku` را یکبار بدون `MRV` و `LCV`، و یکبار با آن‌ها حل کنید. مدت زمانی که برای حل شدن طول می‌کشد چند برابر شد؟ دلیل آن را با درخت جستجو توجیح کنید.

سوال ۸: سودوکوی `Evil.sudoku` را یکبار با `mycsp` و یکبار با `pycsp` حل کنید (گزینه‌ی `real-time` خاموش و بقیه‌ی گزینه‌ها روشن باشند). چه اتفاقی افتاد؟ برای بهبود کتابخانه‌ای که نوشتیم چه راهکاری پیشنهاد می‌دهید؟

سوال ۹: بعضی سودوکوها را آسان و بعضی را سخت تلقی می‌کنیم. به نظر شما تفاوت آن‌ها در چیست؟ از آنجایی که همه‌ی آن‌ها با سه تایی $(\mathcal{X}, \mathcal{D}, \mathcal{C})$ مشخص می‌شوند، کدام یک از این سه تایی درجه‌ی سختی آن‌ها را مشخص می‌کنند؟

توضیحات تکمیلی

- پاسخ به تمرین‌ها باید به صورت انفرادی انجام شود. در صورت مشاهده تقلب، نمره صفر لحاظ خواهد شد.
- پاسخ خود به سوالات که به صورت **سوال** مشخص شده‌اند را در قالب یک فایل PDF به صورت تایپ شده به فرمت `AI_P2Q_student-number.pdf` به همراه دو فایل `sudoku.py` و `mycsp.py` در قالب یک فایل فشرده با فرمت `AI_P2_student-number.zip` در سامانه کورسز آپلود کنید (توجه کنید نوشتن گزارش الزامی است).
- در صورت هرگونه سوال یا ابهام از طریق ایمیل parsa2201@aut.ac.ir با تدریس یاران در ارتباط باشید. همچنین خواهشمند است در متن ایمیل به شماره دانشجویی خود نیز اشاره کنید.
- همچنین می‌توانید از طریق تلگرام نیز با آیدی `@parsa22000` در تماس باشید و سوالاتتان را مطرح کنید.
- این پروژه **تحويل آنلاین یا حضوری از تمام دانشجویان** خواهد داشت و تسلط کافی به سورس کد برنامه ضروری است. بخشی از نمره به صورت ضریب به تسلط شما وابسته است.
- ددلاین این پروژه **۵ اردیبهشت ۱۴۰۴ ساعت ۲۳:۵۹** است.
- بودجه‌ی تاخیر همه‌ی پروژه‌ها در کل ۷ روز می‌باشد و به ازای هر روز دیرتر تحويل دادن ۲۰ درصد از نمره‌ی **کل پروژه** کم خواهد شد. بنابراین بودجه‌ی تاخیر خود را مدیریت کنید.